

Comprehensive Technical Manual: End-to-End AWS CI/CD for Spring Boot on ECS

1. Introduction: The Evolution from Manual to Automated Deployment

Traditional deployment workflows for Spring Boot applications often rely on manual intervention, a process that is inherently prone to error and architectural inconsistency. In a manual lifecycle, a developer must execute three distinct stages: local Docker image creation, manual authentication and upload to the Amazon Elastic Container Registry (ECR), and the manual refreshing of Amazon Elastic Container Service (ECS) tasks to pull the updated image.

The infrastructure requirements of modern software development necessitate a shift away from these manual processes to mitigate several core challenges:

- **Developer Overload:** Engineers are forced to divert focus from feature engineering to repetitive infrastructure management.
- **Repetitive Manual Tasks:** Every minor code revision requires a full, manual build-push-deploy cycle.
- **Operational Risk:** Manual tagging and deployment increase the probability of version mismatch and deployment downtime.

To resolve these inefficiencies, we implement a CI/CD pipeline. **Continuous Integration (CI)** automates the build and containerisation phase—compiling code and pushing verified images to ECR. **Continuous Deployment (CD)** orchestrates the delivery of these artifacts to the hosting environment, ensuring that every commit to the primary branch is automatically reflected in the live environment.

2. Architectural Blueprint

The following architecture defines a robust, automated pipeline utilizing AWS-native services to manage the application lifecycle from source commit to production execution.

AWS Service	Function within the CI/CD Lifecycle
GitHub	Acts as the authoritative source control provider and trigger for the pipeline.
AWS CodeBuild	Executes the CI phase, following <code>buildspec.yml</code> instructions to compile the JAR, build the Docker image, and push to ECR.
AWS ECR	Provides a secure, private Docker registry for storing immutable container images with unique version tags.

AWS ECS	The production hosting environment, utilizing AWS Fargate (serverless) to execute the containerised Spring Boot application.
AWS CodePipeline	The central orchestrator that manages the state and transition of artifacts between Source, Build, and Deploy stages.

3. The Spring Boot Application Layer

The application utilized for this deployment is a Spring Boot service providing RESTful CRUD operations for "course" objects. To eliminate external latency variables and database configuration overhead during the CI/CD demonstration, the application utilizes in-memory collections for data persistence.

Containerisation is achieved via a `Dockerfile` located in the project root. This file ensures environment parity across the pipeline and production.

```
FROM openjdk:17
WORKDIR /app
COPY target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The Dockerfile specifies OpenJDK 17 as the base image, establishes `/app` as the working directory, and exposes port 8080. It is designed to copy the compiled JAR file from the `target` directory, which is generated during the build phase.

4. Continuous Integration Phase 1: AWS ECR and Build Configuration

The initial CI step involves the creation of a private repository in Amazon ECR. The repository should be named "Java registry" and resides within the `ap-south-1` region.

The Buildspec Configuration

The `buildspec.yml` file is the foundational instruction set for AWS CodeBuild. It must be placed in the root of the source code. Note that the `mvn clean install` command is mandatory to generate the JAR artifact required by the Dockerfile.

```
version: 0.2

phases:
  pre_build:
    commands:
      - echo Logging in to Amazon ECR...
      - REPOSITORY_URI=YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/java-registry
      - aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-
stdin $REPOSITORY_URI
      - COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
```

```

    - IMAGE_TAG=${COMMIT_HASH:=latest}
build:
  commands:
    - echo Build started on `date`
    - echo Compiling the Java application...
    - mvn clean install
    - echo Building the Docker image...
    - docker build -t $REPOSITORY_URI:latest .
    - docker tag $REPOSITORY_URI:latest $REPOSITORY_URI:$IMAGE_TAG
  post_build:
    commands:
      - echo Build completed on `date`
      - echo Pushing the Docker images to ECR...
      - docker push $REPOSITORY_URI:latest
      - docker push $REPOSITORY_URI:$IMAGE_TAG
      - echo Writing image definitions file...
      - printf '[{"name":"Java-registry","imageUri":"%s"}]' $REPOSITORY_URI:$IMAGE_TAG >
        imagedefinitions.json
  artifacts:
    files: imagedefinitions.json

```

Phase Analysis:

1. **Pre-build:** The environment initialises the ECR session and extracts the first seven characters of the GitHub commit hash to generate a unique `IMAGE_TAG`.
2. **Build:** The service executes `mvn clean install` to produce the application JAR. Subsequently, the Docker image is built and tagged with both 'latest' and the unique commit hash.
3. **Post-build:** Both image tags are pushed to ECR. Crucially, the process generates the `imagedefinitions.json` file. This artifact serves as the mandatory "handshake" between CI and CD, informing AWS CodePipeline exactly which image URI to deploy to the ECS container named "Java-registry."

5. Continuous Integration Phase 2: AWS CodeBuild Configuration

The "course project" in AWS CodeBuild must be linked to the GitHub repository. During configuration, ensure the project is set to use the `buildspec.yml` file from the source code.

IAM Policy Requirements

The CodeBuild service role requires explicit permissions to communicate with the container registry. The infrastructure requires the attachment of the following policies to the project's service role:

- `AmazonEC2ContainerRegistryFullAccess`
- `AmazonEC2ContainerRegistryPowerUser`

While Full Access provides broad permissions, the source context dictates the inclusion of both to ensure absolute coverage for the build and push operations.

Build Validation

Upon configuration, initialise a manual build. Monitor the logs to ensure the `mvn` compilation and Docker push sequences succeed. Verification is complete once the ECR repository displays two new entries: one with the "latest" tag and one with the unique commit hash.

6. Infrastructure Setup: AWS ECS (Elastic Container Service)

The hosting environment must be provisioned before the pipeline can be finalised.

Task Definition

Create a Task Definition named "course task" using the **Fargate** launch type.

- **Resource Allocation:** Set minimal CPU and Memory (e.g., .5 vCPU and 1 GB RAM).
- **Container Configuration:** Set the container name to "Java-registry." For the Image URI, use the base Repository URI (e.g., `YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/java-registry`). By omitting a specific tag, the service defaults to "latest" during initialisation.
- **Port Mapping:** Explicitly map port 8080 for both the Container Port and the Host Port.

Cluster and Service Deployment

Initialise the "Java cluster" on AWS Fargate. Within this cluster, create a service named "course-service-v2." During the networking configuration, update the **Inbound Rules** of the Security Group to allow **Custom TCP** traffic on **Port 8080** from **Anywhere (0.0.0.0/0)**. This is required for public access to the Swagger UI.

Manual Verification

Once the service reaches an "Active" state, locate the Public IP of the running task. Access the application at `http://<PUBLIC_IP>:8080/swagger-ui.html` to confirm that the Spring Boot CRUD endpoints are reachable.

7. Continuous Deployment: AWS CodePipeline Orchestration

AWS CodePipeline integrates the isolated stages into a unified flow. The creation of "course-pipeline-v2" follows a strict five-step sequence:

1. **Pipeline Settings:** Define the pipeline name and allow AWS to generate the service role.
2. **Source Stage:** Select **GitHub** as the provider, authorise the connection, and specify the "main" branch. Enable **GitHub Webhooks** to ensure the pipeline triggers automatically upon every push.
3. **Build Stage:** Select **AWS CodeBuild** and link the "course project."
4. **Deploy Stage:** Select **Amazon ECS**, targeting the "Java cluster" and "course-service-v2."

5. **Review:** Confirm the configuration and initialise the pipeline.

8. Final Pipeline Validation and Testing

To confirm the architectural integrity of the end-to-end flow, execute a code-level change in the GitHub repository.

1. **Code Modification:** Update the application by adding a new REST endpoint (e.g., `/welcome`) or modifying the visibility of the health check endpoints.
2. **Deployment Trigger:** Commit and push the changes.
3. **Automated Execution:** Monitor the AWS CodePipeline console. The system will automatically detect the commit, trigger the CodeBuild project to generate a new image with a unique hash, and signal ECS to perform a rolling update of the service.
4. **Final Verification:** Once the pipeline status reflects "Succeeded," refresh the Swagger UI at the Public IP address. Confirm that the new endpoint or logic changes are live. This confirms that the transition from code to production is fully automated, requiring zero manual infrastructure intervention.

9. Conclusion

The implementation of this CI/CD pipeline transforms the deployment lifecycle into a streamlined, automated process. By utilising AWS CodeBuild for continuous integration and AWS CodePipeline for deployment orchestration, the developer is liberated from the mechanics of infrastructure, focusing entirely on code quality and feature delivery.

While this manual details a Spring Boot deployment on ECS, the underlying logic—specifically the use of `buildspec.yml` and `imagedefinitions.json`—is the industry standard for AWS automation. This architecture is highly scalable and provides the foundational framework for deploying to other containerised targets, such as Amazon EKS or AWS Elastic Beanstalk.